



Sécurité des communications

JONCOUR Tristan
CDA 3A, TD2, TP4

TP1



Sommaire

Introduction	1
Partie 1 : Server Bob	2
Partie 1 : Client Alice	4
Partie 2 : Server RSA	6
Partie 2 : Client RSA	7
Partie 3 : Interception et modification de messages (MITMPROXY)	8
Partie 4 : Authenticité et intégrité via signature numérique	10
Attaque 1 : remplacement de la clé publique	10
Attaque 2 : usurpation complète	11
Conclusion	12

Introduction

Ce TP aborde les fondamentaux de la sécurité des communications à travers des étapes pratiques et complémentaires, visant à construire une communication chiffrée, authentifiée et robuste face aux attaques.

Partie 1 : Server Bob

1. Établir un secret partagé avec l'algorithme de Diffie-Hellman

```
# -----
# --- Question 1 : Établissement du secret partagé via Diffie-Hellman ---
# Bob génère une clé privée, calcule sa clé publique, puis reçoit celle d'Alice.
# Il utilise sa clé privée et la clé publique d'Alice pour calculer un secret partagé.

# 1. Génération de la paire de clés Diffie-Hellman côté Bob
premier = 0xFFFFFFFFB
generateur = 2
cle_privee_bob = secrets.randbelow(premier)
cle_publicque_bob = pow(generateur, cle_privee_bob, premier)

# Création du socket serveur TCP
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as serveur:
    serveur.bind((hote, port))
    serveur.listen()
    print("Serveur Bob en attente de connexion...")

    connexion, adresse = serveur.accept()
    with connexion:
        print(f"Connecté à {adresse}")

        # Réception de la clé publique d'Alice
        cle_publicque_alice = int(connexion.recv(4096).decode())

        # Envoi de la clé publique de Bob à Alice
        connexion.sendall(str(cle_publicque_bob).encode())

        # Calcul du secret partagé (même clé de session des deux côtés)
        secret_partage = pow(cle_publicque_alice, cle_privee_bob, premier)
        #secret_partage = (pow(cle_publicque_alice, cle_privee_bob, premier) + 1) % premier # Cas d'erreur
        secret_partage_octets = secret_partage.to_bytes(16, byteorder='big')
```

2. Utilisation du secret partagé pour chiffrer

```
# -----
# --- Question 2 : Déchiffrement de la clé AES ---
# Bob reçoit la clé AES chiffrée par Alice via XOR avec le secret partagé.

donnees = connexion.recv(4096)
cle_aes_chiffree = donnees[:16] # Clé AES chiffrée (16 octets)
hash_recu = donnees[16:48] # SHA-256 de la clé AES (32 octets)

# Déchiffrement de la clé AES avec le secret partagé
cle_aes = bytes([c ^ s for c, s in zip(cle_aes_chiffree, secret_partage_octets)])
```

3. Vérification de l'intégrité

```
# -----
# --- Question 3 : Vérification d'intégrité avec SHA-256 ---
# Bob vérifie si la clé AES reçue n'a pas été altérée.

if hashlib.sha256(cle_aes).digest() == hash_recu:
    print("Intégrité confirmée : clé AES correcte.")
else:
    print("Intégrité NON confirmée : clé AES corrompue !")
```

4. Confirmation de la clé par retour de message

```
# -----  
# --- Question 4 : Confirmation par retour de message ---  
# Bob envoie un message chiffré avec la clé AES pour confirmer qu'il l'a bien reçue.  
  
message = b"CBON".ljust(16, b'\x00')  
message_chiffre = bytes([m ^ k for m, k in zip(message, cle_aes)])  
  
connexion.sendall(message_chiffre)  
print("Message de confirmation envoyé à Alice.")
```

Partie 1 : Client Alice

1. Établir un secret partagé avec l'algorithme de Diffie-Hellman

```
# -----  
# --- Question 1 : Établissement du secret partagé via Diffie-Hellman ---  
# Alice génère une clé privée, calcule sa clé publique, envoie celle-ci à Bob,  
# reçoit celle de Bob, puis calcule un secret partagé commun.  
  
# 1. Génération de la paire de clés Diffie-Hellman côté Alice  
premier = 0xFFFFFFFFB  
generateur = 2  
cle_privee_alice = secrets.randbelow(premier)  
cle_publique_alice = pow(generateur, cle_privee_alice, premier)  
  
# Connexion au serveur Bob  
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client:  
    client.connect((hote, port))  
  
    # Envoi de la clé publique d'Alice  
    client.sendall(str(cle_publique_alice).encode())  
  
    # Réception de la clé publique de Bob  
    cle_publique_bob = int(client.recv(4096).decode())  
  
    # Calcul du secret partagé  
    secret_partage = pow(cle_publique_bob, cle_privee_alice, premier)  
    secret_partage_octets = secret_partage.to_bytes(16, byteorder='big')
```

2. Utilisation du secret partagé pour chiffrer

```
# -----  
# --- Question 2 : Génération et chiffrement de la clé AES ---  
# Alice génère une clé AES, puis la chiffre avec le secret partagé (XOR).  
  
cle_aes = os.urandom(16)  
cle_aes_chiffree = bytes([k ^ s for k, s in zip(cle_aes, secret_partage_octets)])
```

3. Vérification de l'intégrité

```
# -----  
# --- Question 3 : Génération du hash SHA-256 de la clé AES ---  
# Alice génère le hash de la clé AES en clair pour permettre à Bob de vérifier son intégrité.  
  
hash_aes = hashlib.sha256(cle_aes).digest()  
  
# Envoi de la clé AES chiffrée + hash  
client.sendall(cle_aes_chiffree + hash_aes)
```

4. Confirmation de la clé par retour de message

```
# -----  
# --- Question 4 : Réception et vérification du message de confirmation ---  
# Alice reçoit un message chiffré de Bob et le déchiffre avec  
# la clé AES pour confirmer que Bob a bien reçu la bonne clé.  
  
message_chiffre = client.recv(4096)  
message_dechiffre = bytes([c ^ k for c, k in zip(message_chiffre, cle_aes)]).rstrip(b'\x00')  
  
if message_dechiffre == b"CBON":  
    print("\n Alice confirme que Bob a bien la bonne clé AES.")  
else:  
    print("\n Erreur : la confirmation n'est pas valide.")
```

Partie 2 : Server RSA

```
from flask import Flask, request, jsonify # Framework web et gestion JSON
import rsa # Bibliothèque de chiffrement RSA

# Étape 1 : Création de l'application Flask
application = Flask(__name__)

# Étape 2 : Génération de la paire de clés RSA (clé publique et clé privée)
cle_publique, cle_privée = rsa.newkeys(512) # 512 bits pour test local (éviter >245 octets)

# Étape 3 : Définition de la route pour fournir la clé publique au client
@app.route('/get_key', methods=['GET'])
def fournir_cle_publique():
    return jsonify({
        'n': cle_publique.n, # Modulus RSA
        'e': cle_publique.e # Exposant public
    })

# Étape 4 : Définition de la route pour recevoir un message chiffré
@app.route('/message', methods=['POST'])
def recevoir_message():
    donnees = request.get_json() # Récupération des données JSON du client
    message_chiffre_hex = donnees['message'] # Le message est reçu sous forme hexadécimale
    message_chiffre = bytes.fromhex(message_chiffre_hex) # Conversion hexadécimal -> binaire

    try:
        # Étape 5 : Déchiffrement du message avec la clé privée RSA
        message_clair = rsa.decrypt(message_chiffre, cle_privée)
        return jsonify({'message_clair': message_clair.decode() }) # Renvoi du texte en clair au client
    except Exception as erreur:
        return jsonify({'erreur': str(erreur) }), 400 # Gestion des erreurs de déchiffrement

# Étape 6 : Lancement du serveur Flask sur le port 5000
if __name__ == '__main__':
    application.run(port=5000)
```

```
(venv) cybertrist@cyberspace:~/Documents/Securité des communications/Partie 2$ p
ython serveur_rsa.py
* Serving Flask app 'serveur_rsa'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment.
Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [02/May/2025 15:04:38] "GET /get_key HTTP/1.1" 200 -
127.0.0.1 - - [02/May/2025 15:04:38] "POST /message HTTP/1.1" 200 -
```

Partie 2 : Client RSA

```
import requests # Pour faire les requêtes HTTP
import rsa      # Pour chiffrer avec la clé publique

# -----
# Adresse du serveur Flask
url_serveur = "http://127.0.0.1:5000"

# -----
# Étape 1 : Récupération de la clé publique RSA depuis le serveur
reponse = requests.get(f"{url_serveur}/get_key")
cle_publique_data = reponse.json() # n et e (composants de la clé publique RSA)

# Reconstruction de l'objet clé publique RSA
n = cle_publique_data['n']
e = cle_publique_data['e']
cle_publique = rsa.PublicKey(n, e)

# -----
# Étape 2 : Préparation et chiffrement du message
message = "Bonjour depuis le client RSA."
message_chiffre = rsa.encrypt(message.encode(), cle_publique)

# -----
# Étape 3 : Envoi du message chiffré au serveur via POST
reponse = requests.post(f"{url_serveur}/message", json={
    'message': message_chiffre.hex() # Encodage hexadécimal pour JSON
})

# -----
# Étape 4 : Affichage de la réponse (texte clair déchiffré côté serveur)
if reponse.status_code == 200:
    print("Réponse du serveur :", reponse.json()['message_clair'])
else:
    print("Erreur côté client :", reponse.text)
```

```
(venv) cybertrist@cyberspace:~/Documents/Securité des communications/Partie 2$ python3 client_rsa.py
Réponse du serveur : Bonjour depuis le client RSA.
(venv) cybertrist@cyberspace:~/Documents/Securité des communications/Partie 2$
```

Partie 3 : Interception et modification de messages (MITMPROXY)

L'objectif de cette partie est de démontrer les limites du chiffrement asymétrique RSA, lorsqu'il n'est pas accompagné de mécanismes d'authentification ou d'intégrité. Pour cela, nous utilisons mitmproxy comme attaquant en homme du milieu (Man-In-The-Middle) afin de modifier les échanges entre un client et un serveur.

Ce qui se passe, le client et le serveur communiquent via le protocole HTTP.

Le client chiffre un message à l'aide de la clé publique du serveur, et ce dernier le déchiffre ensuite avec sa clé privée. Afin de simuler une attaque de type Man-In-The-Middle, le client est configuré pour passer par un proxy HTTP local (mitmproxy) écoutant sur `http://127.0.0.1:8888`. Un script nommé `mitm_attack.py` est utilisé pour intercepter et modifier à la volée les requêtes POST contenant le message chiffré. Le serveur, qui n'effectue aucune vérification d'authenticité ni d'intégrité, tente tout de même de déchiffrer le message, ce qui le rend vulnérable à une altération non détectée par un tiers malveillant.

```
from mitmproxy import http # Permet d'intercepter et modifier les requêtes HTTP
import json                # Pour lire et modifier le corps JSON de la requête
import base64              # Pour encoder les données binaires en base64

# -----
# Fonction appelée automatiquement à chaque requête HTTP interceptée
# -----
def request(flow: http.HTTPFlow) -> None:
    # On cible uniquement les requêtes POST envoyées à l'URL "/message"
    if flow.request.method == "POST" and "/message" in flow.request.pretty_url:
        try:
            # -----
            # Décodage du corps de la requête JSON (envoyée par le client)
            # -----
            data = json.loads(flow.request.content)

            print("[MITM] Requête originale interceptée :", data)

            # -----
            # CORRUPTION du contenu : modification du message chiffré
            # -----
            # Injection de 64 octets à zéro (faux message), encodés en base64
            data["message"] = base64.b64encode(b"\x00" * 64).decode()

            # -----
            # Remplacement du corps de la requête avec les nouvelles données corrompues
            # -----
            flow.request.text = json.dumps(data)

            print("[MITM] Requête modifiée envoyée au serveur.")
        except Exception as e:
            print("[MITM] Erreur de modification :", e)
```

Le serveur, en revanche, échoue à déchiffrer le message, une exception de déchiffrement. Le message chiffré a été modifié avec succès par l'attaquant, sans que ni le client ni le serveur ne puissent vérifier sa légitimité.

Un attaquant peut donc intercepter et modifier un message chiffré sans être détecté.

Partie 4 : Authenticité et intégrité via signature numérique

L'objectif maintenant est d'assurer non seulement la confidentialité (déjà vue en Partie 2), mais aussi l'authenticité de l'émetteur (le client), et l'intégrité du message.

Pour cela, le client signe numériquement le message avant chiffrement, et le serveur vérifie la signature avec la clé publique du client.

Attaque 1 : remplacement de la clé publique

Dans cette première attaque, l'attaquant intercepte une requête parfaitement légitime, contenant un message correctement chiffré et signé par le client.

Sans toucher au message ni à la signature, il remplace uniquement la clé publique du client par une autre, qu'il génère lui-même.

Lorsque le serveur reçoit la requête, il tente de vérifier la signature à l'aide de cette fausse clé publique.

Évidemment, la signature ne correspond pas, car elle n'a pas été faite avec la clé privée associée. Le serveur détecte donc une signature invalide et rejette le message.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\trist\OneDrive - Université Bretagne Sud\Documents\Securité des communications\Partie 4> python serveur_rsa.py
Génération des clés RSA du serveur...
Clés générées.
Serveur lancé sur http://127.0.0.1:5800
* Serving Flask app 'serveur_rsa'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production
WSGI server instead.
* Running on http://127.0.0.1:5800
Press CTRL+C to quit
Envoi de la clé publique du serveur.
127.0.0.1 - - [14/May/2025 22:44:39] "GET /get_key HTTP/1.1" 200 -

Requête POST reçue.
Payload reçu : {'message': 'bTzAzt+yZ9348K62fd2h8N05E9fMnpYkUM8lDYDX4j5z2G5Qq/W90czWnnpaf/TpPHLZ
D/HGc3AhP97z8G7zj3eNbp1DOUBRhm8BefNcThx8PHCPqzVXoz3W1ZnDzuxWVaRm80TdBnt8zWLLtjmt1uE13W9GgJlKp
TjfkXkXoJCVl8G6FjWwH-jXviVH5Gq481fneEo11tVmwjxTg9uCU8R8Q8W/hq/Vb/avVrLCDHczcSsBsLu9+xy3bZi4shx6a
4nmgd1VCX9shru1qEHd4rxw81qum1JhTdA17MbJ17JEboVVIHkIn5XD6FzSBqfx3vgwCIAxGp8FwG==', 'signature':
'Ga80yUPUCesD7l81/Dyq6yZXVfmlpR99B11Cx7GmLFCvCR9mFTR+BP4Kys2q48tt8J7Gxoz21JkPvdyQMlQjpxrzCGL
Bz1SL3KScjBqRbubSa+VvPKLmPujXscZC++xvj9jmuVCh1q1xKy33mpAR0/PUs8+2U3gpe2aT67Tt6ZAUu8A85Jp6C
UQ0vYzOfy7vLFOCGYIBelnxCH/62zpSgdl7VqW9uv1VtwtCm+ulJTX3UWHvTjg814Vb08Ns1Q7z97Lrq5klpe91I31YoX8
4v7iZjzIFhctkZaAe15/pfghu2FV3/neyHx1joP3o1CHt128TxQ==', 'client_pubkey': '-----BEGIN RSA PUBLIC
KEY-----\nMIIBBgKCAQEA17ydoiVEZTxNAtvEX1Gp19r/B2EgeZqLAcVECP6iap/9w1Ulbna\N5saj5wGyQ2Lh9LAwCKEE
39c8G/S6p4IE19VLMlTadgux3r1j6bv4Op14w40Z9az\Nz9MkLJdR9/T7chrX3/BQCCQj/BRj57Hf1S5856/AFQoVYio8Xpy
eoxxtmglSV\N/EQ/Kv8h5e5WQV/21TtXU1nZbX9z1az6fJeuQEClpVzaw8B1cNv2e8Jy5t8V\NfPv8r1Hd3BZQWpCv
pb7oafz1TeP8eJ991zS85sa6q1m1TL08wdkva1HtHj9W18\Nf1D+ns5YfWDCszuhE8h/Vz5n6uQ/QA84w1DQAQ8\N-----
\N
ND RSA PUBLIC KEY-----\n'}
Clé publique du client reconstruite.
Message déchiffré : Message authentique signé par le client légitime.
Signature invalide.
127.0.0.1 - - [14/May/2025 22:44:41] "POST /message HTTP/1.1" 400 -

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\trist\OneDrive - Université Bretagne Sud\Documents\Securité des communications\Partie 4> python
attaque1_remplacement_cle_publique.py
Génération de la vraie paire de clés du client légitime...
Clés RSA du client légitime générées.
Message clair : Message authentique signé par le client légitime.
Récupération de la clé publique du serveur...
Clé publique du serveur récupérée.
Clé publique du serveur reconstruite.
Message chiffré.
Signature du message générée avec la vraie clé du client.
Génération d'une fausse clé publique pour simuler une usurpation...
Clé publique de l'attaquant prête.
Payload d'attaque construit (avec clé publique falsifiée).
Envoi du payload falsifié au serveur...
Réponse du serveur : {'message_clair': None, 'status': 'Signature invalide'}
PS C:\Users\trist\OneDrive - Université Bretagne Sud\Documents\Securité des communications\Partie 4>
```

Cette attaque échoue, ce qui démontre que la signature numérique garantit bien l'intégrité du message et protège partiellement contre l'usurpation d'identité, à condition que la clé publique soit elle-même fiable.

Attaque 2 : usurpation complète

Dans cette seconde attaque, l'attaquant ne se contente pas de modifier la clé publique : il remplace l'ensemble du contenu de la requête.

Il génère sa propre paire de clés RSA, rédige un message frauduleux, le chiffre avec la clé publique du serveur, puis le signe avec sa propre clé privée.

Enfin, il envoie au serveur ce message chiffré, sa signature, ainsi que sa clé publique personnelle.

Le serveur, ne disposant d'aucun mécanisme permettant de vérifier l'identité réelle de l'expéditeur, considère que la signature est valide (car elle correspond bien à la clé publique reçue). Il accepte donc la requête comme si elle venait du client d'origine.

```
Windows PowerShell
Clés générées.
Serveur lancé sur http://127.0.0.1:5000
* Serving Flask app 'serveur_rsa'
* Debug modes: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
Envoi de la clé publique du serveur.
127.0.0.1 - - [14/May/2025 22:49:56] "GET /get_key HTTP/1.1" 200 -

Requête POST reçue.
Payload reçu : {'message': 'OQsa7Y9epLnM1/6hQxHjTrKlNrsyAZj453WxYsaS+NfLgZcbnaOZFGp0csgFqmw7Rc+7VZDs
lMdeYSGbZZTdP99aP1WjyLBSIOrtSRXn2uuvsgTw9f7n78d1y/wTTJKTZpskhWvDa3WgD+x3yaX1nJVDv9C3e9cvFghLzjXkLdF
0UwSdpXtH39RZaeBvSQNSTPG/QLHSsQTYLARFoj+EKMc/jnd32TyBAHpeBAK6XseyF8/KLcS2G+EuMIZ+4Y3Ba0brFSC7kat9g
Yw5xh91RCjthNrs7Q08rz6zLRANwzhVwqVEBcUeOBQKSPzSLdtPYJewASQ3oM124s1FG88Q==', 'signature': 'mGINEo7
7L1Fk59rLTdbKsm+SQqGAQg7bk1Ygo2ZcLSU2vBLHLYw6LLsLj8DF4b72VfALHG1KashLAV1f4mRbOE+8qpBE0v+UdL9D2P
E10835vX1wB3nalcWMMXw1nc91yCrf4/71kyJ0/dg6/3q6d38q2hmbFC36S2ry3yTrdJ22Q1cFhclWP30b76TBBYxMhFL
R2/Q324K5J1IN8M6wHP1f1tjY3M4GvSna9F+e8+sSkRWLLjly7quo9J9TJ4sbvZ81S2tuykzFVSp7IB1HcpF0x9YaB1LD4jpnQva
T83JLH9MujHIVNj+x0wkuhW28x2CdghbTVAQ==', 'client_publickey': '-----BEGIN RSA PUBLIC KEY-----\nmMIIBCg
KCAQEAA4cKtKo+NB80GLUXV2xkdT3YzKAmbw/JVXakI9Dux4aNTvunBQMT\NaRGHj+uqeyih21CGqHdFT+Vc3ZdU5q7BfJMH4Jv
f71apHnU9hAdGjGYxNR1y3rx6JntRFLxVzWg5QwGAQqh5Vv6sZu3E3IU4W7b3wqNSaY+6VwqpuqJGUgMDuiwF4Fu2N\Nv13eA1
Bj/EPT0DGr4clqXbVL7/fq/dHzGSwLLw8og+MRhyrabHu5DcIdq8RLN/m\N81F7ktU36+MV/qPqtKuSI0hX8nYAGA2+pgDphR1
Yez4g5hZABDu2mYpXufh7z2/nuaKBRuJ6zVGq7h8Rk4L8kg26xwx4Nj6HrwIDAQAB\n-----END RSA PUBLIC KEY-----\n
'}
Clé publique du client reconstruite.
Message déchiffré : Ceci est un message frauduleux envoyé par Tristan.
Signature VALIDE.
127.0.0.1 - - [14/May/2025 22:49:57] "POST /message HTTP/1.1" 200 -

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\trist\OneDrive - Université Bretagne Sud\Documents\Securité des communications\Partie 4> python
attaque2_usurpation_identite.py
Génération des clés RSA de l'attaquant...
Clés de l'attaquant générées.
Message frauduleux préparé : Ceci est un message frauduleux envoyé par Tristan.
Récupération de la clé publique du serveur...
Clé publique du serveur récupérée.
Clé publique du serveur reconstruite.
Message frauduleux chiffré.
Signature frauduleuse générée avec la clé de l'attaquant.
Payload frauduleux prêt à être envoyé.
Envoi du message frauduleux au serveur...
Réponse du serveur : {'message_clair': 'Ceci est un message frauduleux envoyé par Tristan.', 'status': 'Sig
nature valide'}
PS C:\Users\trist\OneDrive - Université Bretagne Sud\Documents\Securité des communications\Partie 4>
```

Cette attaque met en évidence une faille majeure, la signature numérique seule ne permet pas d'authentifier l'émetteur, puisqu'elle ne garantit pas à qui appartient la clé publique transmise.

Conclusion

Ce TP m'a permis de comprendre les limites du chiffrement RSA seul, l'importance de la signature numérique pour garantir l'intégrité, et la nécessité d'un système d'authentification fiable pour se protéger des attaques de type Man-In-The-Middle.